UNIVERSIDAD DE LAS FUERZAS ARMADAS – ESPE
Department of Computer Science
Software Engineering
36862 – Object-Oriented Programming

UNIT 1 PROJECT · SaborCode

Project Inspection Report

Team	ByteWorks
Members	Andrés Pachacama, Carlos Parra, Matias Paucar
NRC	36862
Instructor	Edison Lascano, PhD
Date	October 30, 2026
Academic period	SII-2026 (September 2026 – February 2027)

1. Scope and Methodology

The inspection was performed on the cloned repository (file OOP36862-SaborCode.rar). For each artifact in the rubric, the delivered document was reviewed and compared with the source code. In addition, the following actions were carried out:

- Reading of the definition, list of objects, SRS and Product Backlog PDFs, and of the UML diagrams (PNG).
- Compilation of the SaberCodeV0.5.0 project with *javac* (JDK 21) and execution of the 90 tests with JUnit 4.13.2.
- Execution of the console application with 11 input scenarios (normal flow and invalid inputs).
- Review of the generated files: menu_saborcode.json and reporte_orden_*.csv.

Folder	Content
01DefinitionOfProblem	ProjectDefinition.pdf, README.md (empty)
02Requirements	ListObjects.pdf, SRS.pdf, README.md (empty)
03ProductBacklog	ProductBacklog.pdf, README.md (empty)
04UMLDiagrams	Class and use case diagrams (.png and .vpp), README.md (empty)
05UnitTest	NetBeans screenshot with test results, README.md (empty)
06Code	NetBeans project SaberCodeV0.5.0 (src, test, build, nbproject, JSON and CSV)
07Other	Only README.md (empty)

2. Evaluation by Artifact

2.1 Problem Definition

Requirement: self-explanatory.

Strengths:

- It includes an overview, problem statement, operational context and performance measures.
- It identifies those affected (diners, waiters, kitchen, owner) and the impact of the problem.

Observations:

- The document describes a **web** solution with QR codes, an admin panel and a kitchen view, but what was delivered is a Java console application with none of that.
- The performance measures (QR access rate, order processing time) are not computed by the system.
- Minor typo on the cover ("Project't Name").

2.2 List of Possible Objects

Requirement: at least 10 well-defined objects.

Strengths:

- 22 objects are proposed, grouped by module (diner, kitchen, administration, services).

Observations:

- Only names are listed; no object has attributes, responsibilities or a description, so they are not "well defined".
- Several items are not domain objects but modules or screens: Admin Panel, Kitchen Display System, Restaurant Configuration, Dietary Filter.
- Of the 22 objects, only Table, Dish, Order and Order Item appear in the code. Conversely, the classes ChickenPiece, Drink, Side, ExtraPortion and Tip, which were implemented, are not on the list.

2.3 SRS IEEE 830

Requirement: completeness.

Strengths:

- It includes purpose, scope, product perspective, user classes and 9 functional requirements with priority and owner.

Observations:

- Mandatory sections of the standard are missing: 1.3 definitions and acronyms, 1.4 references, 1.5 overview, 2.2 product functions, 2.4 constraints, 2.5 assumptions and dependencies.
- Section 3 (specific requirements) is incomplete: external interfaces (user, hardware, software), non-functional requirements (performance, security, usability) and quality attributes are missing.
- Requirements RF001-RF009 are a copy of the user stories; they have no inputs, processing, outputs or acceptance criteria.
- The traceability matrix repeats the backlog table instead of linking requirements to classes, use cases and tests.
- The SRS does not describe the actual functionality of the program (tables, chicken pieces, tip, CSV).

2.4 Product Backlog

Requirement: at least 9 features that compute something (-1 for each one missing).

Strengths:

- There are 9 stories in the "As a... I want... so that..." format, with priority and owner.

Observations:

- None of the 9 stories describes a calculation: they are all about management (CRUD, roles), navigation (QR) or notification. Each story counts as one observation (9).
- What the program does compute (subtotal, tip, total to pay, available tables by capacity) is not in the backlog.
- There is no estimation (story points), acceptance criteria or sprint planning.
- Suggested stories that do compute something: order total with tip, bill split between diners, best-selling dishes, sales per day, average service time, table occupancy.

2.5 UML Class Diagram

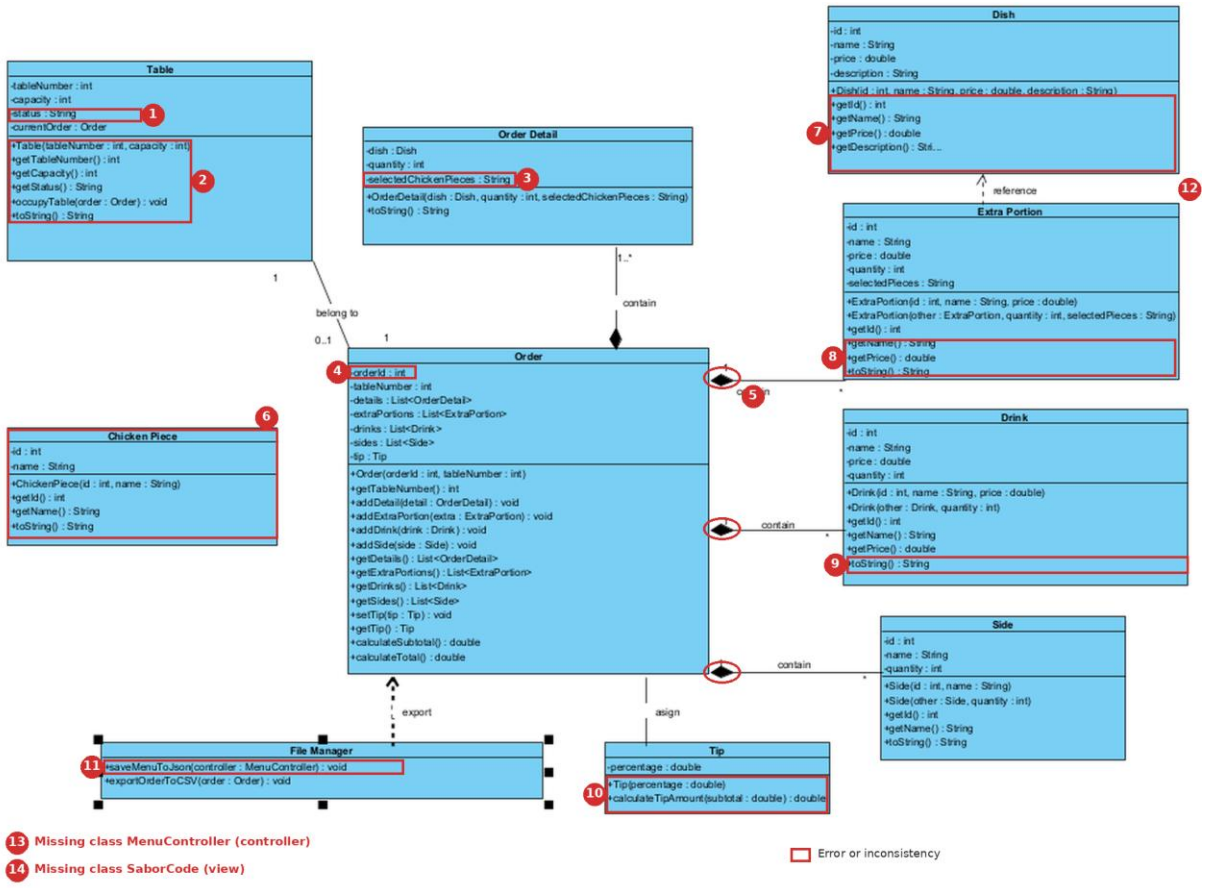
Requirement: classes and relationships.

Strengths:

- It includes 10 classes with attributes, methods, visibility and types.
- It uses association (Table-Order), composition (Order-OrderDetail) and dependency (ExtraPortion-Dish, FileManager-Order) with multiplicities.

Observations:

- MenuController (controller package) and SaborCode (view package) do not appear; the diagram does not show the MVC architecture used.
- ChickenPiece is isolated, with no relationship to OrderDetail or ExtraPortion, although the code uses it to choose chicken pieces.
- FileManager is only related to Order, but in the code it also depends on MenuController.
- The relationships between Order and Drink/Side/ExtraPortion are modeled as composition; in the code the order items are created by copying the menu items, so this is debatable and not justified.
- Inheritance is not used: Dish, Drink, ExtraPortion and Side repeat id, name and price, which would fit in a common abstract class (for example, MenuItem).
- There are several differences with the code; they are detailed in the clean code tables.



No.	Marked error
1	Table.status is a String ("Libre"/"Ocupada") instead of a dedicated type.
2	Table: releaseTable() is missing, although it exists in the code.
3	OrderDetail: the attribute selectedChickenPieces does not match the code (selectedPieces); the second constructor, getSubtotal() and the getters are missing.
4	Order: orderId does not match the code (id) and getId() is missing.
5	Composition (filled diamond) between Order and Drink, ExtraPortion and Side, although the items are copied from the menu.
6	ChickenPiece is isolated: it has no relationship with OrderDetail or ExtraPortion, and its setters are missing.
7	Dish: the setters that exist in the code are missing.
8	ExtraPortion: getQuantity(), getSelectedPieces(), getSubtotal() and one constructor are missing.
9	Drink: getQuantity() and getSubtotal() are missing.
10	Tip: getPercentage() is missing.
11	FileManager receives a MenuController, but the relationship is not drawn.
12	Dish, Drink, ExtraPortion and Side repeat attributes and there is no inheritance.
13	The MenuController class (controller package) is missing.
14	The SaborCode class (view package) is missing.

2.6 UML Use Case Diagrams

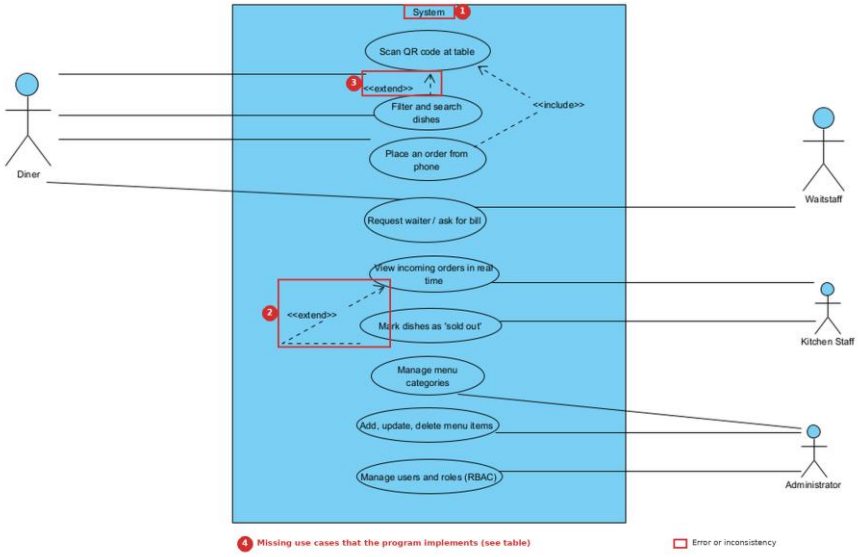
Rubric requirement: use cases and actors.

Strengths:

- Four actors (Diner, Waitstaff, Kitchen Staff, Administrator) and 10 use cases that correspond to the 9 stories.
- It uses <<include>> and <<extend>>.

Observations:

- There is a dangling <<extend>> relationship (dashed line that does not start from any use case) next to "View incoming orders" and "Mark dishes as sold out".
- "Filter and search dishes" appears as an extension of "Scan QR code", which makes no functional sense.
- The system boundary is only called "System" instead of "SaborCode".
- The use cases the program actually performs are missing: choose table, choose chicken pieces, add extra portions, give a tip and generate the CSV report.
- Only one diagram is delivered, although the rubric refers to diagrams in plural.



No.	Marked error
1	The system boundary is only called "System" and does not have the project name.
2	Dangling «extend» relationship: the dashed line does not start from any use case.
3	Filter and search dishes extends Scan QR code, which makes no functional sense.
4	The use cases the program implements are missing: choose table, choose chicken pieces, extra portions, drink and sides, tip and CSV report.

2.7 Clean Code

Requirement: list of pitfalls and inconsistencies with the UML.

Strengths:

- Descriptive class and method names in English; private attributes; use of model/controller/view packages; use of try-with-resources when writing files.

Observations:

- There are 15 bad practices and 11 inconsistencies with the class diagram. The code screenshots supporting each one are in Appendix A (CC-01 to CC-15 and UML-01 to UML-11).

Bad practices (clean code)

Evid.	Location	Problem	Recommendation
CC-01	SaborCode.main	The main method has about 270 lines with the whole flow, input reading and business logic inside a switch.	Split it into methods (chooseTable, takeOrder, checkout) and move the logic to the controller.
CC-02	SaborCode.main	Hard-coded values: dishes 1 to 4 include a chicken piece, extra 8 is "Presa de Pollo", texts "1 al 10".	Add an attribute (for example requiresChickenPiece) and compute the ranges from the lists.
CC-03	SaborCode	nextInt() without validation; the program crashes with letters or out-of-range numbers.	Create a readInt(min, max) method with try/catch and retry.
CC-04	SaborCode	Four almost identical loops search by id in different lists.	Methods findDishById, findDrinkById, etc. in MenuController.
CC-05	Dish, Drink, ExtraPortion, Side	Repeated attributes and methods (id, name, price, getSubtotal, copy constructor).	Use inheritance or an interface (MenuItem / OrderItem).
CC-06	Table	The status is a String ("Libre"/"Ocupada").	Use a TableStatus enum.
CC-07	Order, Tip	Money handled with double; the CSV shows 6.325.	Use BigDecimal or round to 2 decimals.
CC-08	Tip, quantities	The tip and the quantities are not validated as positive.	Validate in the model and throw IllegalArgumentException.
CC-09	FileManager	It is in model but depends on controller and uses System.out; it mixes layers.	Move it to a utils/persistence package and return results instead of printing.
CC-10	FileManager	JSON built by hand through concatenation and never read.	Use Gson or Jackson to save and load.
CC-11	Order.getSides	Sides are stored but do not appear in the CSV or in the empty-order check.	Include them consistently.
CC-12	SaborCode	Order id: System.currentTimeMillis() % 1000000; it can repeat.	Use a counter or a UUID.
CC-13	Several classes	NetBeans template comments, repeated @author, mix of Spanish and English in texts.	Clean up comments and choose one language for messages.
CC-14	Dish, Drink	Inconsistent style: one-line getters in some classes and block getters in others.	Apply the IDE formatter.
CC-15	Tests	Tests that fail on purpose mixed with valid ones.	Separate them or use assertNotEquals so they state the correct behavior.

Inconsistencies between the class diagram and the code

Evid.	Class	In the UML	In the code
UML-01	MenuController	Does not appear	Exists; holds the whole menu
UML-02	SaborCode (view)	Does not appear	Exists; holds the main flow

Evid.	Class	In the UML	In the code
UML-03	Order	Attribute orderId : int; no method to get it	Attribute id : int with getId(), used by FileManager and the tests
UML-04	OrderDetail	selectedChickenPieces : String; a single constructor	selectedPieces : String; two constructors; getSubtotal() and getters are not in the UML
UML-05	Drink / ExtraPortion	No getQuantity() or getSubtotal()	Both methods exist and are used in Order
UML-06	ExtraPortion	getSelectedPieces does not appear	Exists
UML-07	Dish / ChickenPiece	Only getters	They also have setters
UML-08	Table	No releaseTable()	releaseTable() exists (although the view never uses it)
UML-09	Tip	percentage, calculateTipAmount	It also has getPercentage()
UML-10	ChickenPiece	No relationships	Used to choose the chicken pieces of OrderDetail and ExtraPortion
UML-11	FileManager	Depends only on Order	Also depends on MenuController and all the menu classes

2.8 Test Cases / Unit Tests

Requirement: at least 90 running unit tests.

Strengths:

- The result was reproduced: **Tests run: 90, Failures: 22** (68 pass, 75.56 %), the same as the delivered screenshot.
- 9 test classes with 10 tests each, one for each model class.

Observations:

- 22 tests are written to fail on purpose (names *testCaseN_FailOn...*, with wrong expected values). If the instructor requires all tests to pass, this criterion is not met.
- Many tests only check getters and constructors (for example CheckGallitoDrink, TableCapacityFour); they add little to the verification of the logic.
- There are no tests for MenuController, FileManager (JSON/CSV) or the view.
- Edge cases are not tested: negative quantities, negative tip, non-existent table.
- The project is configured for Java 26 (javac.source=26); it had to be compiled with JDK 21, without changes to the code.

Test class	Tests	Pass	Fail
ChickenPieceTest	10	10	0
DishTest	10	10	0
DrinkTest	10	9	1
ExtraPortionTest	10	6	4
OrderDetailTest	10	3	7
OrderTest	10	8	2
SideTest	10	5	5
TableTest	10	9	1
TipTest	10	8	2

Test class	Tests	Pass	Fail
Total	**90**	**68**	**22**

2.9 JSON Data Files

Requirement: the system is persistent through time.

Strengths:

- A valid JSON is generated with dishes, chicken pieces, extra portions, drinks and sides.

Observations:

- The program **only writes** menu_saborcode.json at startup; the file is never read anywhere in the code. The menu is always loaded from hard-coded values in MenuController.
- Orders, occupied tables and sales are not saved; everything is lost when the program closes.
- The JSON is built by concatenating text, without a library (Gson/Jackson) and without escaping quotes: a name with double quotes would produce an invalid file.
- FileManager is in the model package, but it performs input/output and prints to the console.

2.10 CSV Data Reports

Requirement: generates data reports (at least two reports).

Strengths:

- When the bill is requested, reporte_orden_<id>.csv is created with category, description, quantity, unit price, subtotal and total.

Observations:

- There is only **one type** of report: the detail of an order. At least two are required (for example, daily sales or best-selling dishes).
- Sides are not included in the CSV even though they are part of the order.
- Values are not rounded: the delivered CSV shows "Total con Propina;6.325".
- The file name uses an id based on the system time, which could repeat between runs.

2.11 Execution of Features

Rubric requirement: run every feature (-1 for each one that does not work).

Observations:

- Of the 9 backlog stories, 7 are not implemented and US008 is only partial (8 observations, first table).
- Six scenarios in the second table end in a crash or a wrong result (E3, E4, E6, E7, E10, E11): 6 observations.
- Only one table and one order can be served per run; when the bill is requested the program ends and never releases the table.

Backlog features versus execution

ID	Story	Result	Evidence
US001	Scan QR code	**Not implemented**	There is no QR code; the table is chosen by typing a number
US002	Manage categories	**Not implemented**	Categories are fixed in the code
US003	Dish CRUD with allergens	**Not implemented**	There is no option to add, edit or delete; Dish has no allergens
US004	Search and filter dishes	**Not implemented**	Only the full catalog is shown
US005	Place an order	**Works**	Order with dish, chicken pieces, extras, drink and side (from the console)
US006	Kitchen display (KDS)	**Not implemented**	There is no kitchen view
US007	Mark as sold out	**Not implemented**	There is no availability attribute
US008	Call waiter / request bill	**Partial**	Option 3 computes the bill; the waiter notice is only a printed message
US009	Users and roles (RBAC)	**Not implemented**	There are no users or login

Scenarios executed in the console

#	Input	Result obtained
E1	Table 2; 2x Parrillada Normal (Ala, Pechuga); extra Presa (Pierna); 2x Colada Morada; Ají; 10 % tip	Correct. Subtotal $13.00, tip $1.30, total $14.30; the CSV is generated
E2	View catalog and exit	Correct.
E3	Table 7	Crashes: IndexOutOfBoundsException (SaborCode.java, line 58)
E4	Letter in the quantity ("abc")	Crashes: InputMismatchException (nextInt not validated)
E5	Request the bill with no products	Correct. Shows "Su orden esta vacia"
E6	Tip of -50 %	Logic error: accepts a negative tip, total $0.50 for a $1.00 dish
E7	Table 0	Crashes: IndexOutOfBoundsException
E8	Two orders in the same bill	Correct. They are added up in the bill
E9	Letter in the main menu	Correct. Asks for a valid number
E10	Non-existent chicken piece (9)	Assigns "Pechuga" by default without asking again
E11	Quantity 0	Changes the quantity to 1 without telling the user

Inconsistencies between documents

- The definition and the SRS describe a web system with QR, kitchen and administration; the code is an ordering console for a grill restaurant.
- The list of objects, the backlog and the use case diagram do not mention chicken pieces, extra portions, drinks, sides or tips, which are the core of the code.
- The class diagram does reflect the code, but not the list of objects or the SRS. The artifacts are not traceable to each other.

Priority recommendations:

- Align the backlog with what the system does and rewrite the stories so that each one computes something (totals, sales, occupancy, best-selling dishes).
- Read the JSON at startup and save orders and table status so that data persists between runs.
- Add at least a second CSV report (for example accumulated sales or best-selling products).
- Validate all console inputs to avoid crashes.
- Complete the SRS with the IEEE 830 sections and update the diagrams so they match the code.
- Fill in the READMEs, add a .gitignore (build/, nbproject/private/) and commit from Git with descriptive messages.

Appendix A. Code Evidence (Clean Code)

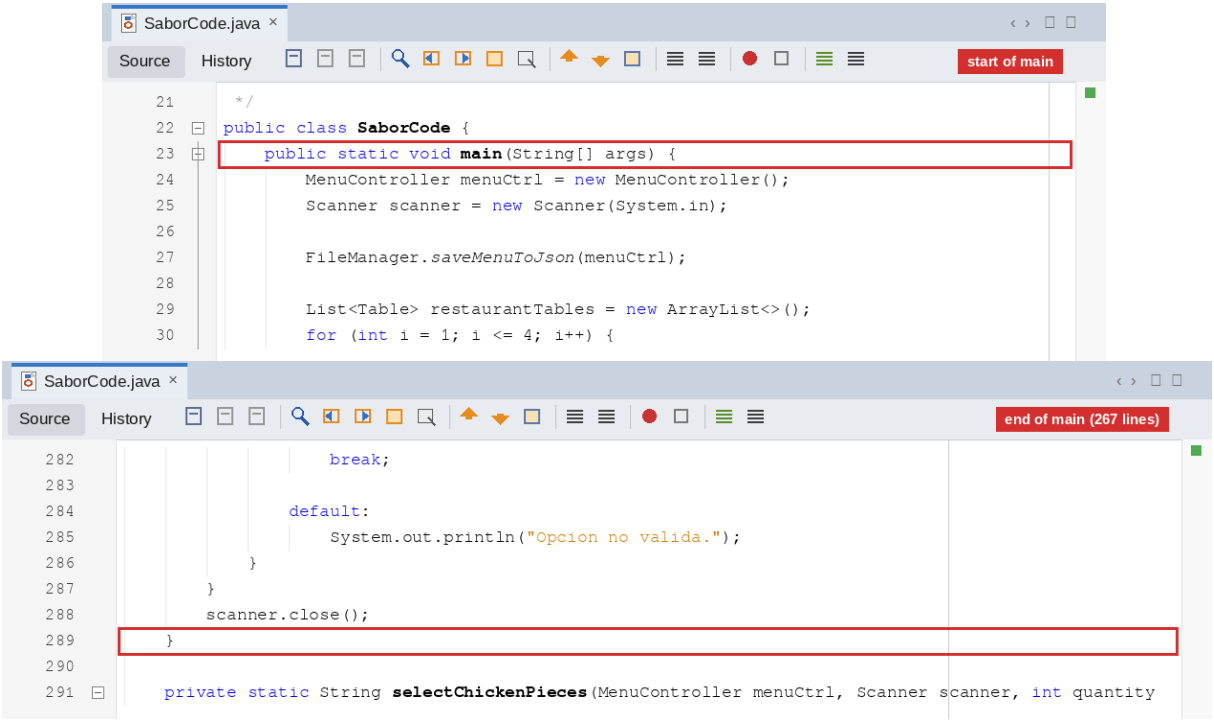
The screenshots were generated from the delivered source code (SaberCodeV0.5.0). The line number is the one in the original file and the lines with the error are marked with a red box.

A.1 Bad practices

CC-01. Main method too long

Location: view/SaborCode.java, lines 23 to 289

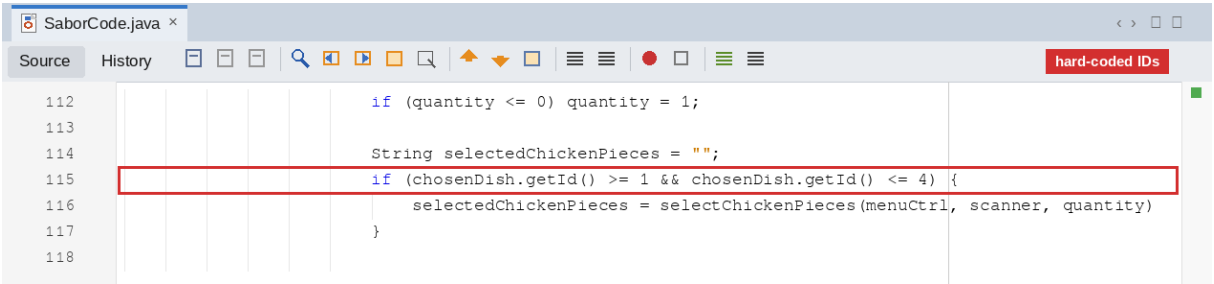
The whole program lives in main: data input, product search, order building and checkout. It is 267 lines in a single method.

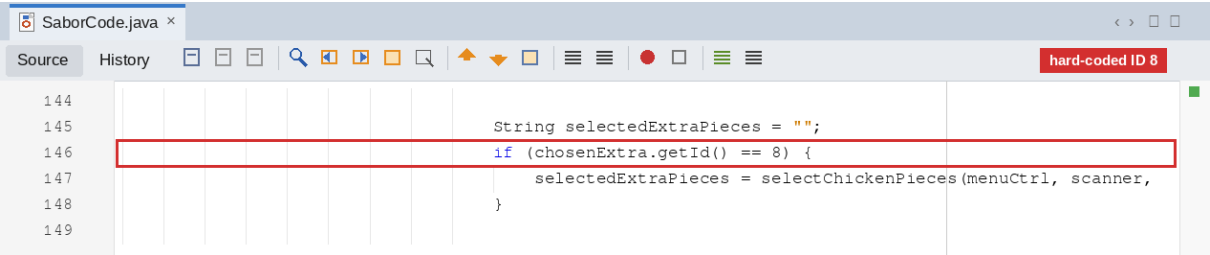


CC-02. Hard-coded values (magic numbers)

Location: view/SaborCode.java, lines 115 and 146

Whether a dish includes a chicken piece depends on its id being between 1 and 4, and the extra chicken portion is identified by id 8. If the menu changes, the program stops working correctly.

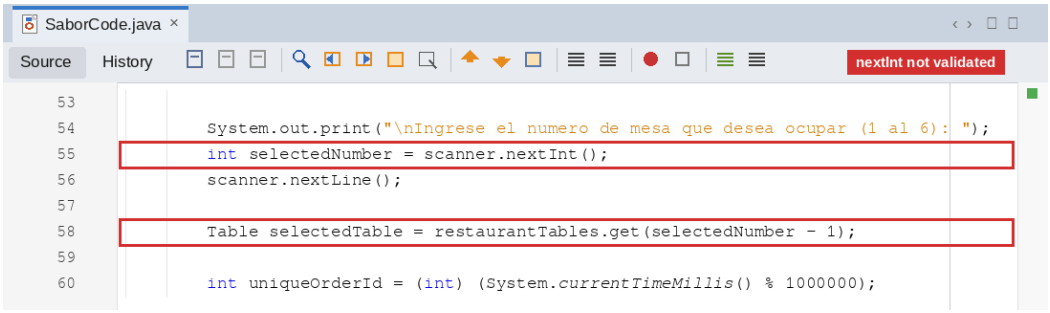




CC-03. Input not validated

Location: view/SaborCode.java, lines 55 and 58

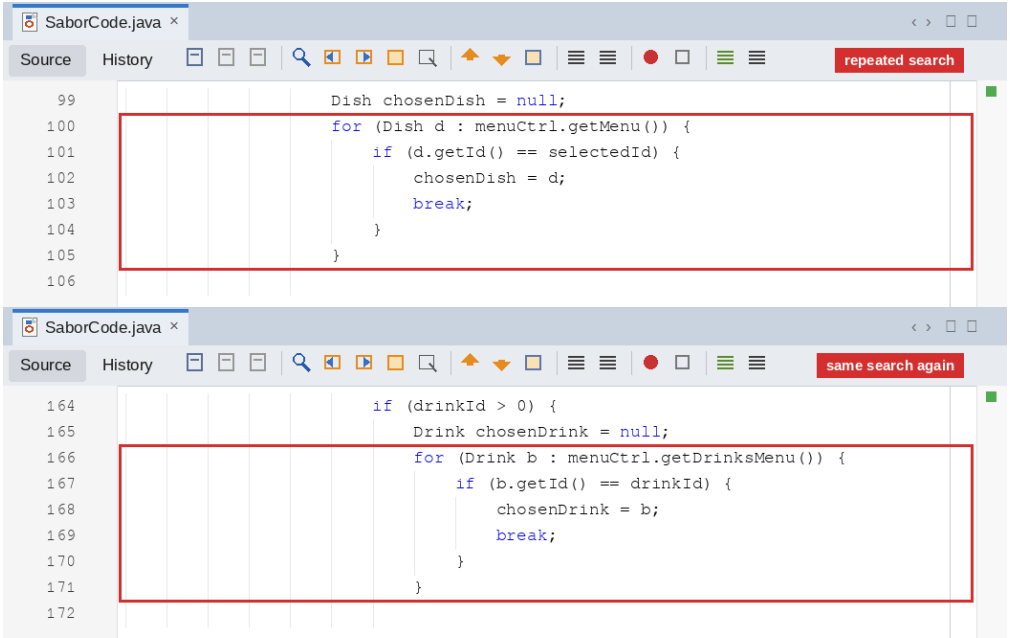
The table number is read with nextInt() and used directly as an index. With 0, 7 or a letter the program crashes (scenarios E3, E4 and E7).



CC-04. Repeated code to search by id

Location: view/SaborCode.java, lines 100-105, 132-137, 166-171 and 198-203

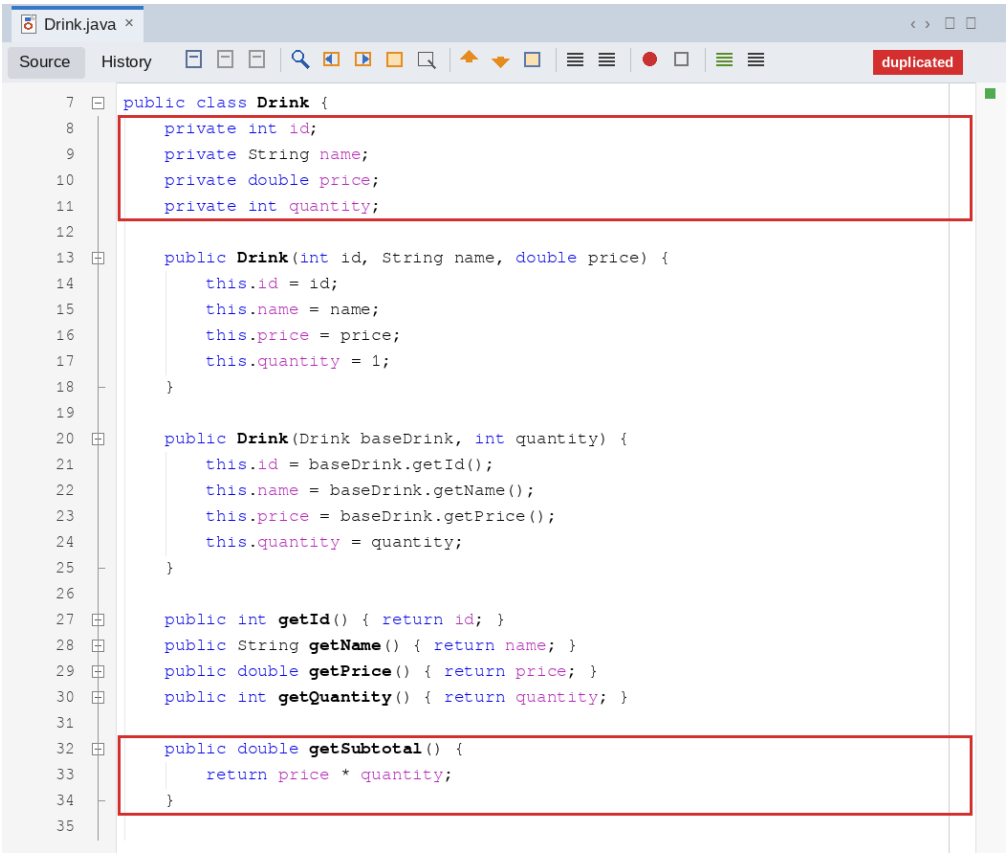
The same search loop is copied for dishes, extras, drinks and sides. It should be a controller method.



CC-05. Classes with duplicated attributes and methods

Location: model/Drink.java and model/ExtraPortion.java

Both classes repeat id, name, price, quantity and getSubtotal(). The same happens with Dish and Side. A common superclass or interface is missing.



ExtraPortion.java ×

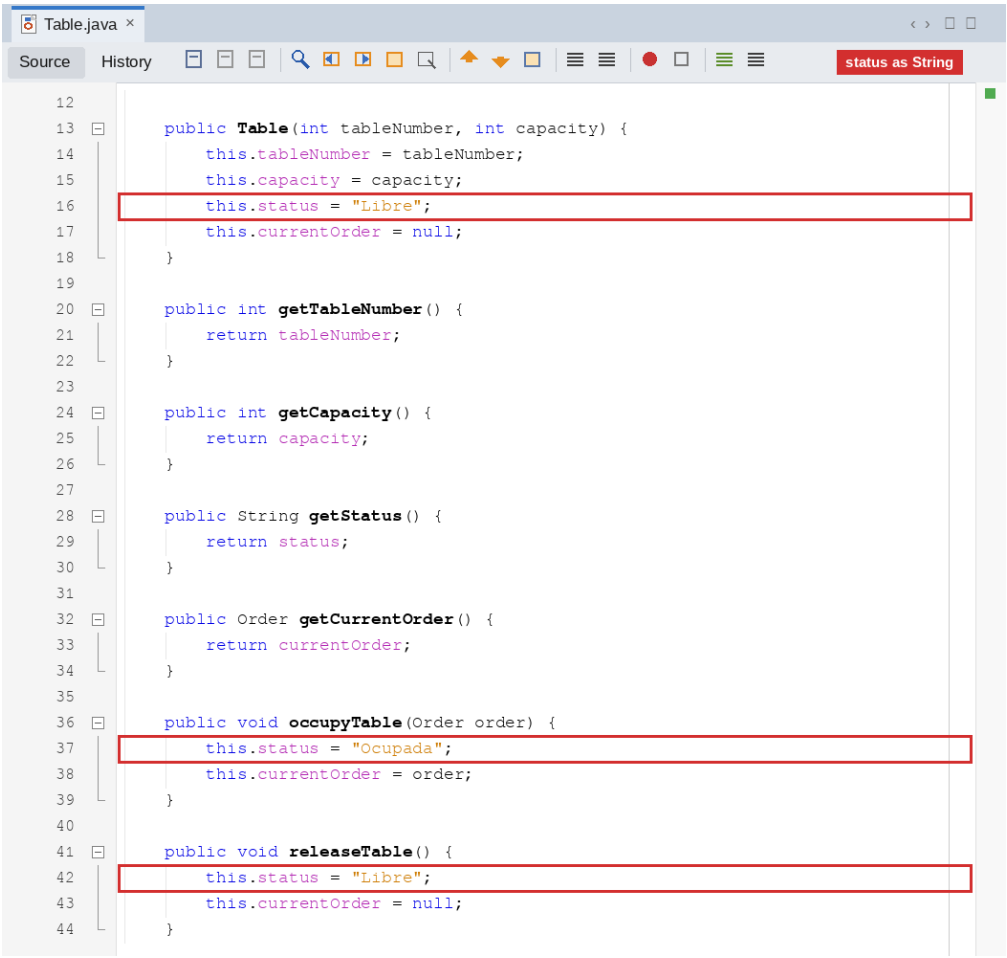
Source History duplicated

```
7  public class ExtraPortion {
8      private int id;
9      private String name;
10     private double price;
11     private int quantity;
12     private String selectedPieces;
13
14     public ExtraPortion(int id, String name, double price) {
15         this.id = id;
16         this.name = name;
17         this.price = price;
18         this.quantity = 1;
19         this.selectedPieces = "";
20     }
21
22     public ExtraPortion(ExtraPortion baseExtra, int quantity) {
23         this.id = baseExtra.getId();
24         this.name = baseExtra.getName();
25         this.price = baseExtra.getPrice();
26         this.quantity = quantity;
27         this.selectedPieces = "";
28     }
29
30     public ExtraPortion(ExtraPortion baseExtra, int quantity, String selectedPieces) {
31         this.id = baseExtra.getId();
32         this.name = baseExtra.getName();
33         this.price = baseExtra.getPrice();
34         this.quantity = quantity;
35         this.selectedPieces = selectedPieces;
36     }
37
38     public int getId() { return id; }
39     public String getName() { return name; }
40     public double getPrice() { return price; }
41     public int getQuantity() { return quantity; }
42     public String getSelectedPieces() { return selectedPieces; }
43
44     public double getSubtotal() {
45         return price * quantity;
46     }
47
```

CC-06. Table status as a String

Location: model/Table.java, lines 16, 37 and 42

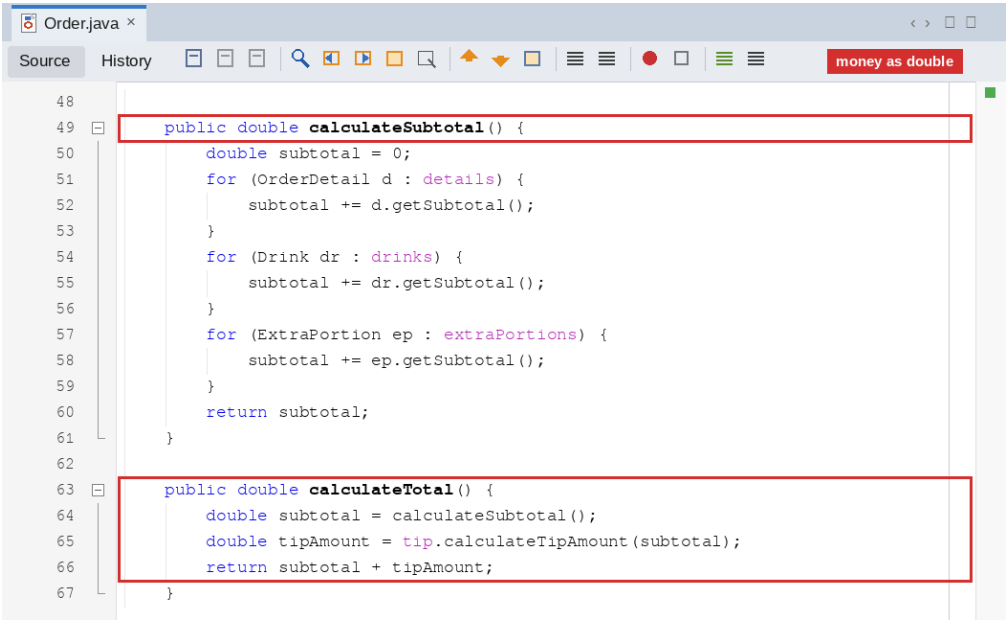
The "Libre" and "Ocupada" states are loose strings; a typo is not detected by the compiler. It should be an enum.



CC-07. Money as double

Location: model/Order.java, lines 49 and 63-66

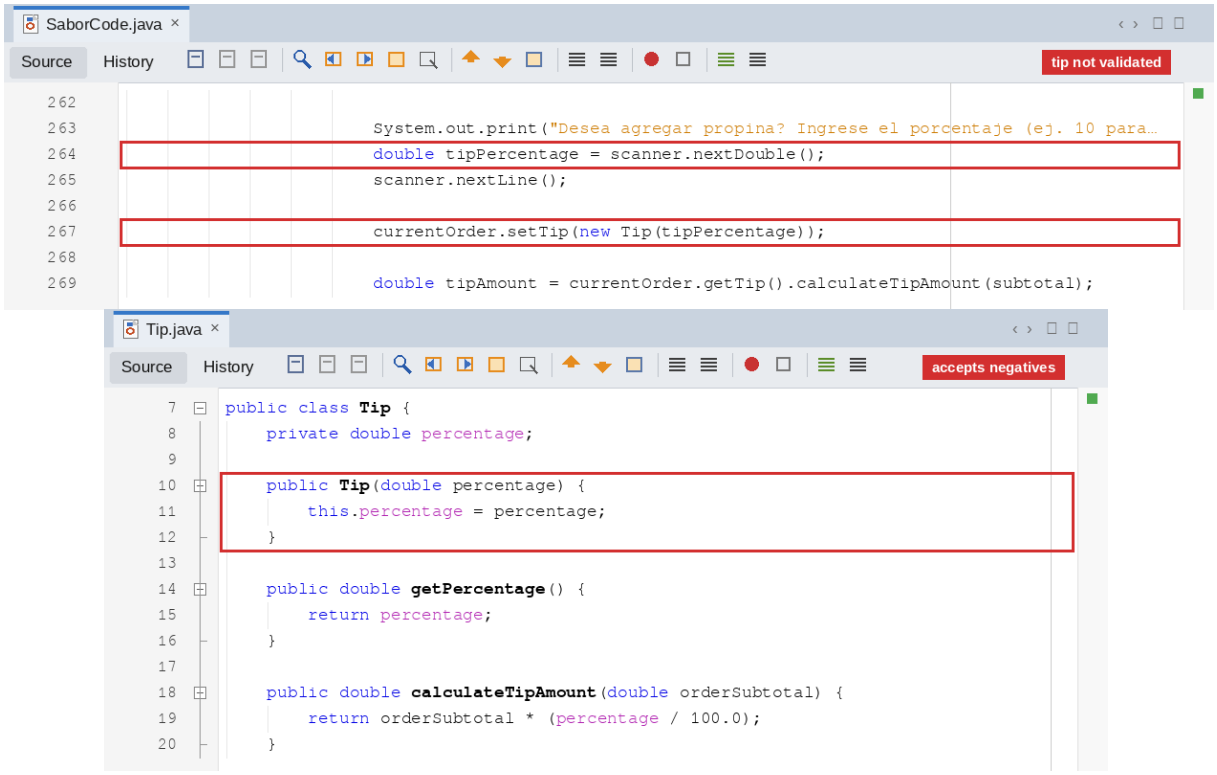
Amounts are added with double and not rounded; that is why the CSV shows 6.325.



CC-08. Tip not validated

Location: view/SaborCode.java, lines 264 and 267; model/Tip.java, lines 10-12

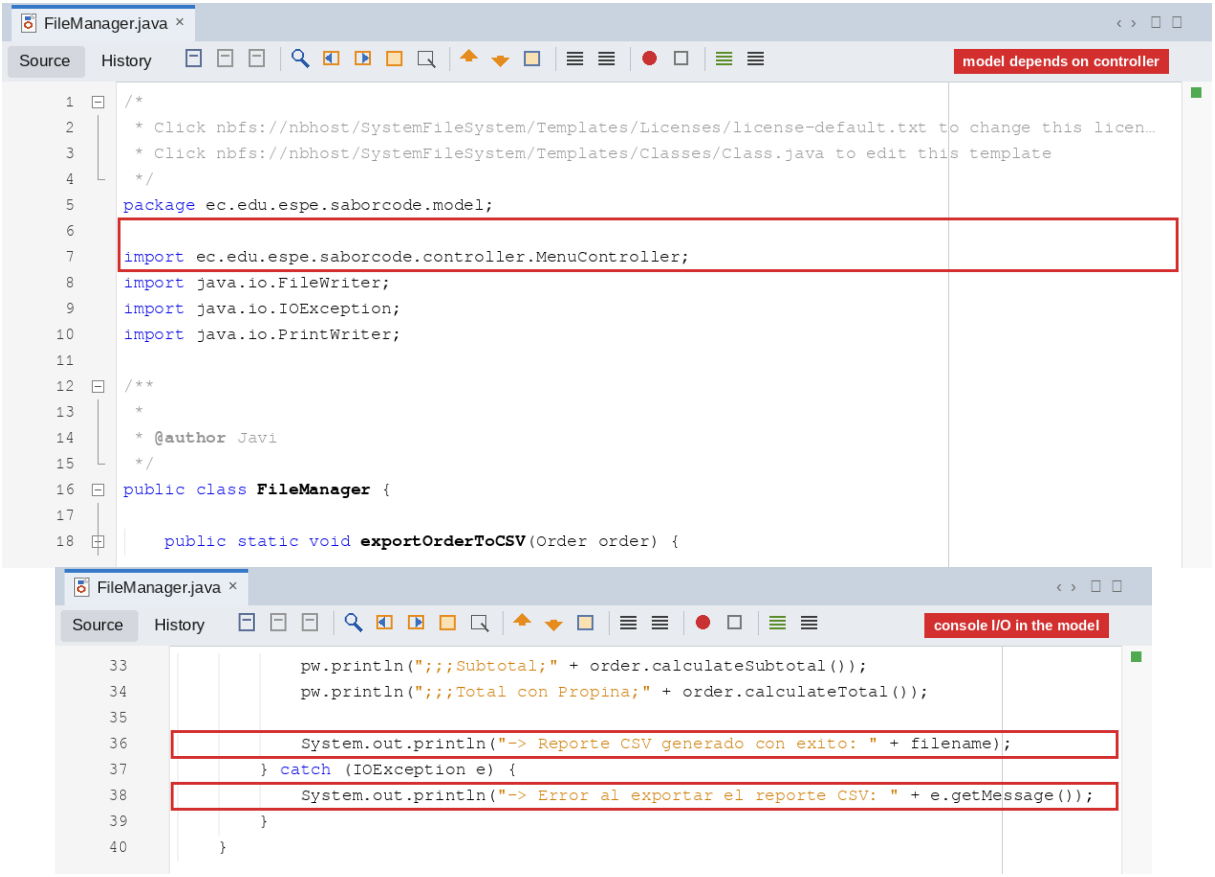
Any percentage is accepted, even a negative one (scenario E6: -50 % tip).



CC-09. Mixed layers in FileManager

Location: model/FileManager.java, lines 7, 36 and 38

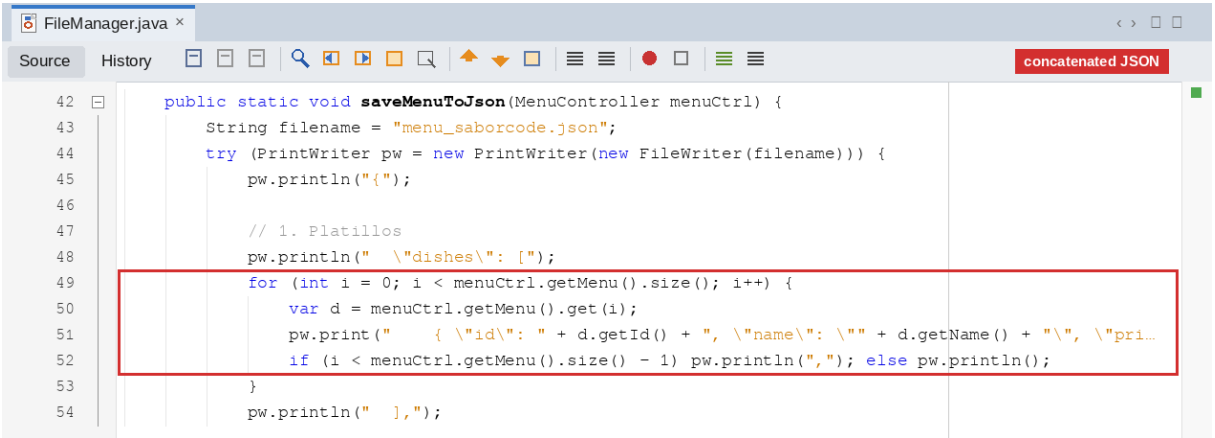
A class in the model package imports the controller and prints to the console.



CC-10. JSON built by hand

Location: model/FileManager.java, lines 49-52

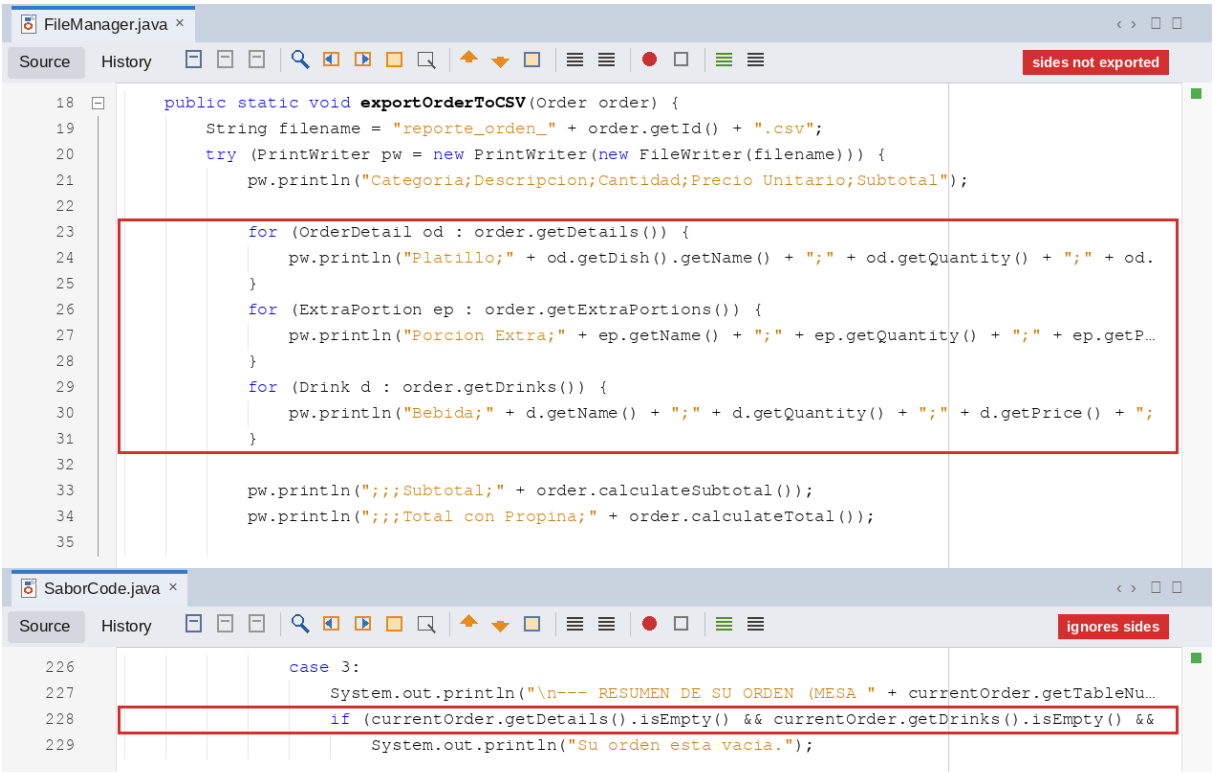
The JSON is built by concatenating text, without escaping quotes and without a library. It is also never read.



CC-11. Sides handled inconsistently

Location: model/FileManager.java, lines 23-31; view/SaborCode.java, line 228

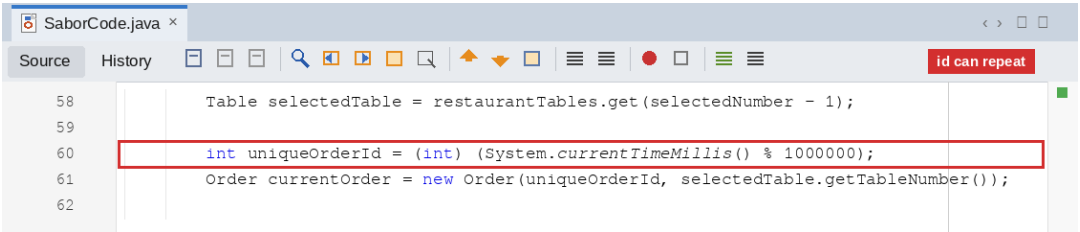
The CSV goes through dishes, extras and drinks, but not sides. The empty-order check does not consider them either.



CC-12. Order id based on the time

Location: view/SaborCode.java, line 60

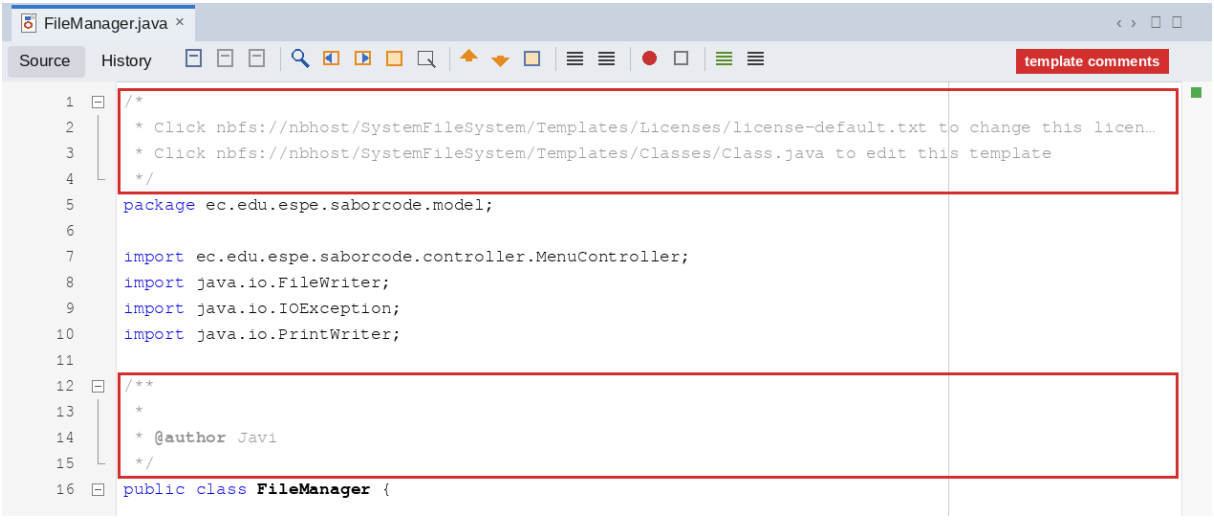
currentTimeMillis() % 1000000 repeats roughly every 1000 seconds and can overwrite a previous report.



CC-13. IDE template comments

Location: model/FileManager.java, lines 1-4 and 12-15

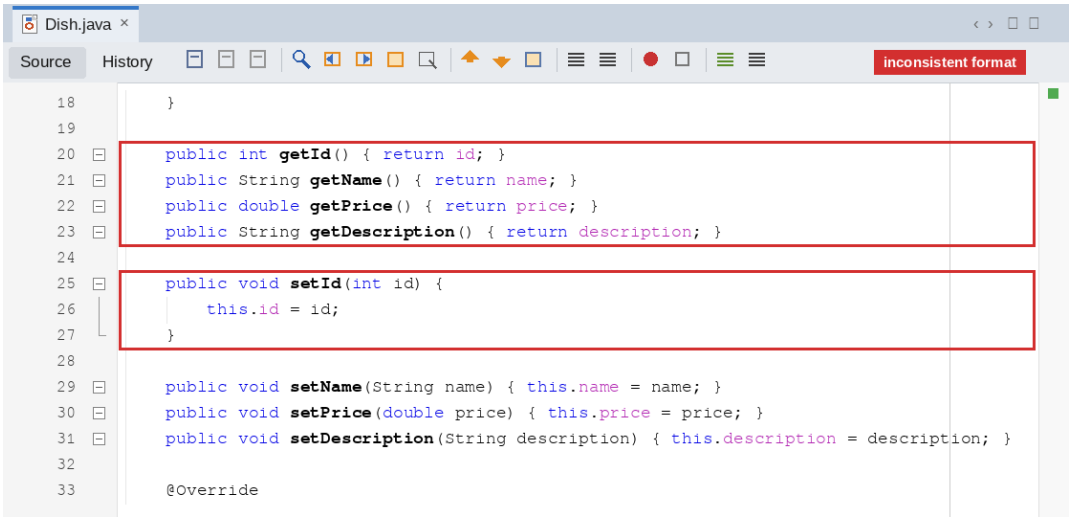
The automatic NetBeans comments and an empty Javadoc were left in.



CC-14. Inconsistent formatting

Location: model/Dish.java, lines 20-27

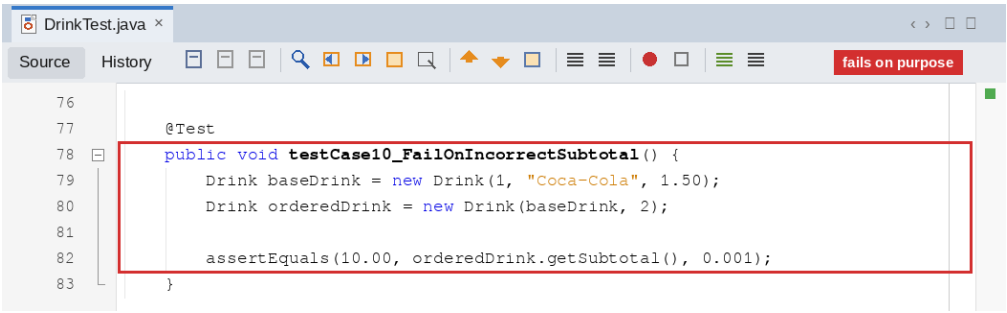
In the same class there are one-line getters, setters split across several lines and others on a single line.



CC-15. Test that fails on purpose

Location: test/.../DrinkTest.java, lines 78-82

2 drinks at $1.50 give $3.00, but the test expects $10.00. There are 22 tests like this in the project.

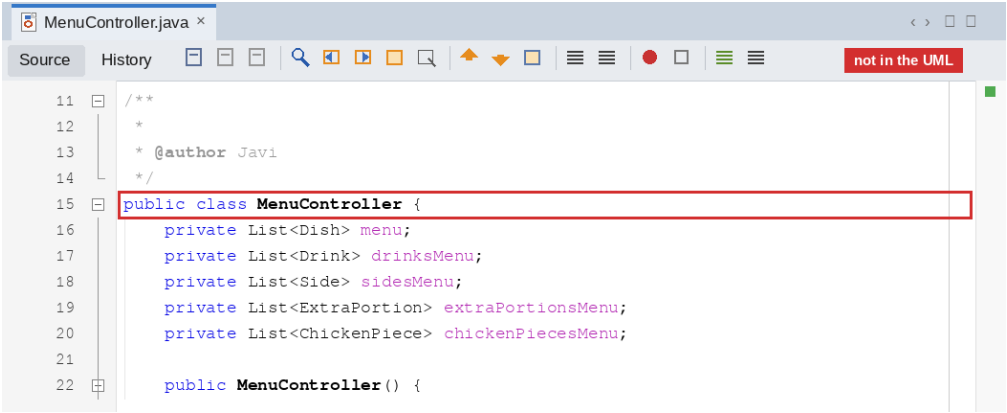


A.2 Inconsistencies with the class diagram

UML-01. MenuController does not appear in the diagram

Location: controller/MenuController.java, line 15

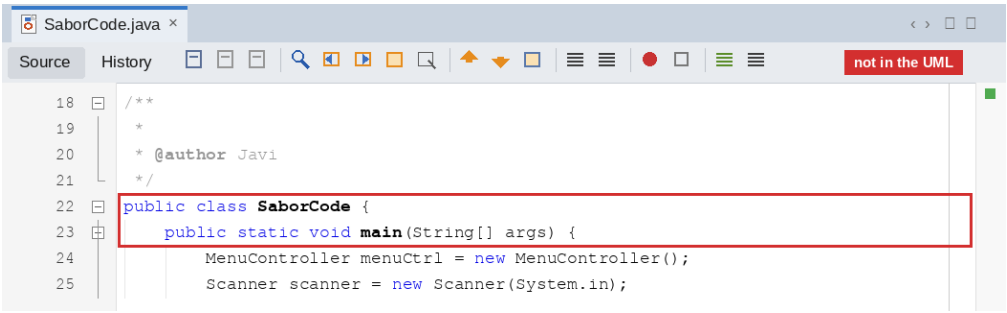
The class exists and holds the whole menu, but it is not in the class diagram.



UML-02. The SaborCode view does not appear in the diagram

Location: view/SaborCode.java, lines 22-23

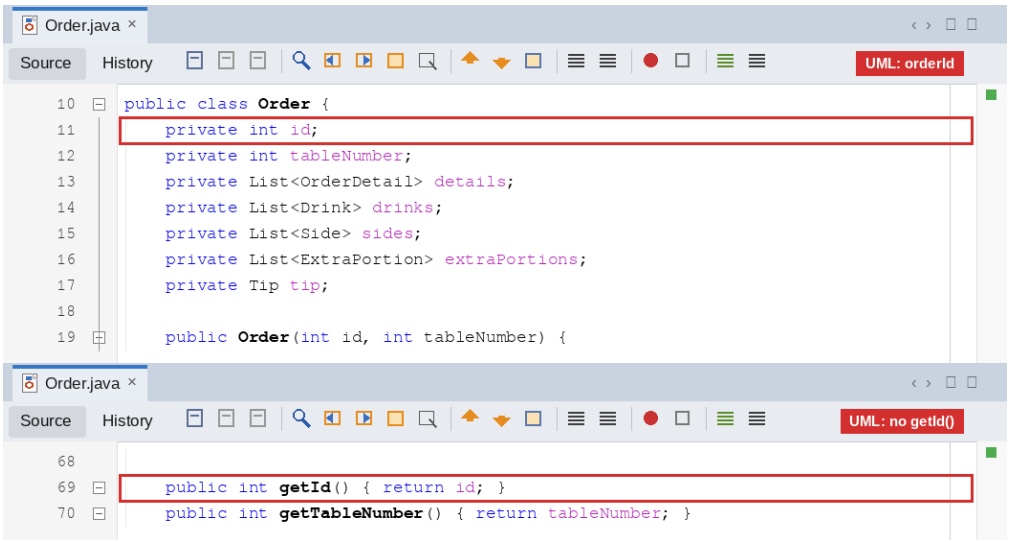
It contains the main method and the whole flow, but it is not modeled.



UML-03. Order: orderId in the UML, id in the code

Location: model/Order.java, lines 11 and 69

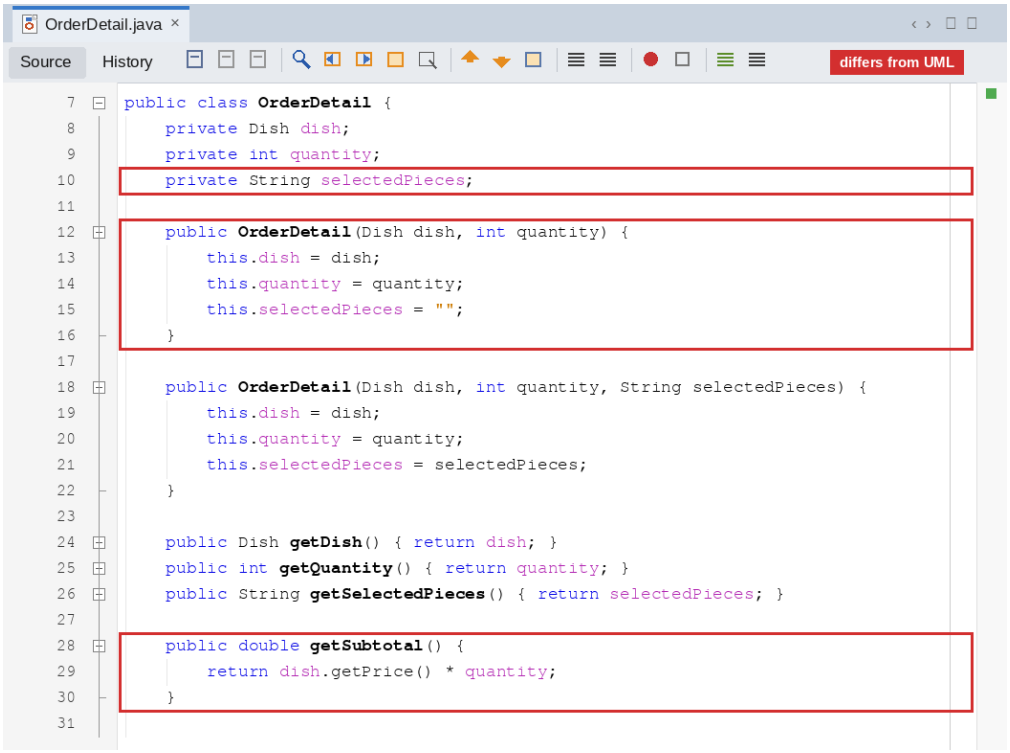
The diagram says orderId and has no getter; the code uses id with getId().



UML-04. OrderDetail differs from the UML

Location: model/OrderDetail.java, lines 10, 12-16 and 28-30

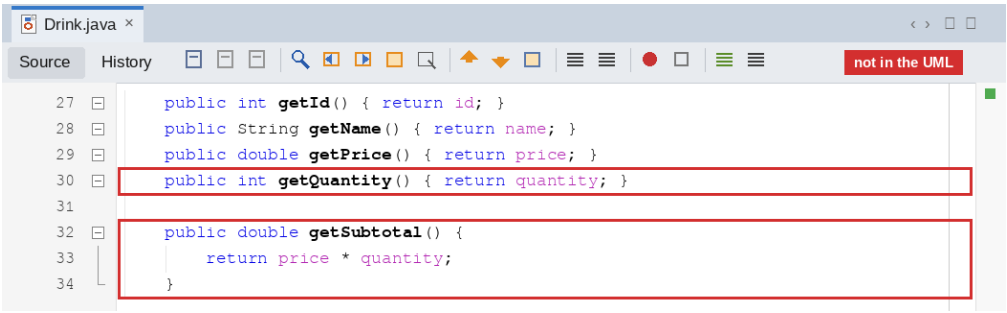
The attribute is called selectedPieces (not selectedChickenPieces), there is a second constructor and getSubtotal() is not in the diagram.



UML-05. Drink: methods missing in the UML

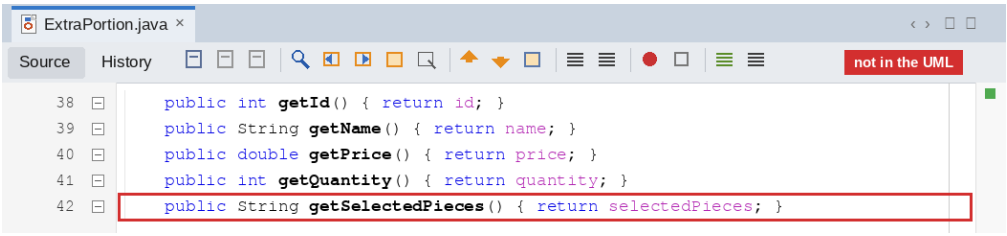
Location: model/Drink.java, lines 30 and 32-34

getQuantity() and getSubtotal() exist and Order uses them, but they are not in the diagram (same in ExtraPortion).



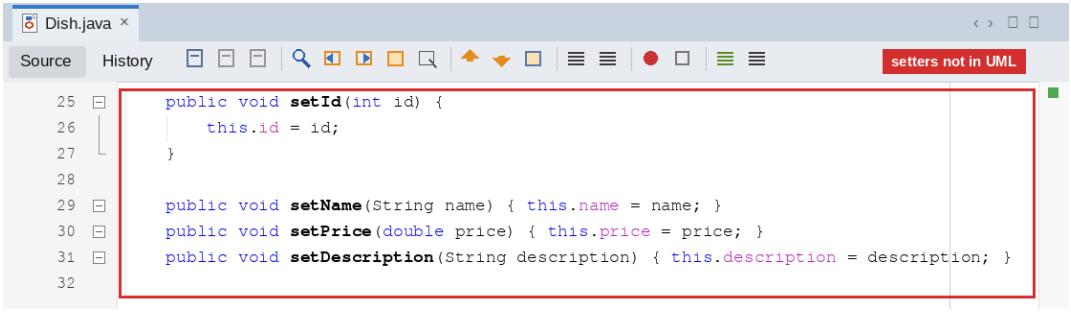
UML-06. ExtraPortion: getSelectedPieces() is not in the UML

Location: model/ExtraPortion.java, line 42



UML-07. Dish and ChickenPiece have setters the UML does not show

Location: model/Dish.java, lines 25-32; model/ChickenPiece.java

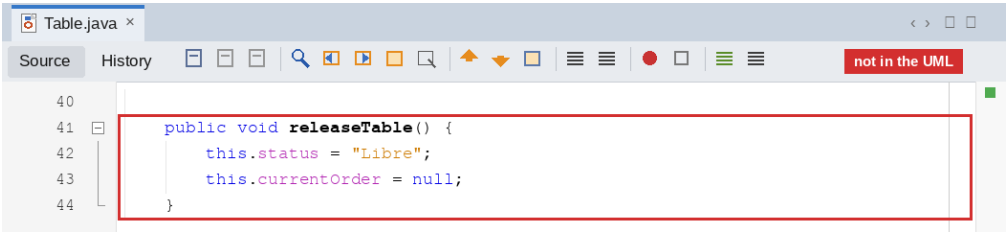




UML-08. Table: releaseTable() is not in the UML

Location: model/Table.java, lines 41-44

In addition, the view never calls it.



UML-09. Tip: getPercentage() is not in the UML

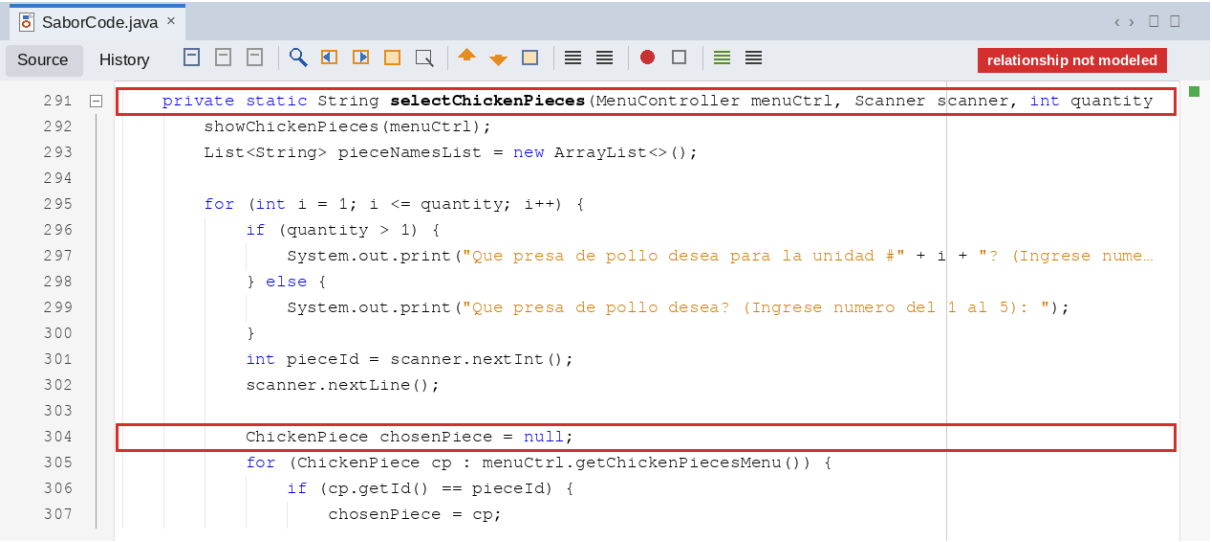
Location: model/Tip.java, lines 14-16



UML-10. ChickenPiece is used, but it is isolated in the UML

Location: view/SaborCode.java, lines 291 and 304

Chicken pieces are chosen with ChickenPiece objects and stored in OrderDetail and ExtraPortion; that relationship was not drawn.



UML-11. FileManager depends on MenuController

Location: model/FileManager.java, line 42

The diagram only shows the dependency with Order.

